Объектно-ориентированное программирование

Классы · Инкапсуляция · Наследование · Полиморфизм · Магические методы · SOLID

Зачем ООП? Проблема, которую оно решает

- Чем больше программа, тем больше «лапши»: функция calculate_user_stats() вызывает process_data(), та — update_database()...
- Данные размазаны по коду: имя пользователя — в одном месте, его счёт — в другом, логика проверки — в третьем
- **ООП группирует связанные данные и функции для работы с ними в одно целое — объект**
- Python и так «всё объект»: числа, строки, списки, функции

print(type(123)) → <class 'int'> · print(type([1, 2])) → <class 'list'>

История ООП: Симула (1967) и Smalltalk (1972)

Банковский счёт на языках-основателях ООП

Simula 67

```
! Первый язык с классами.
! Создан для моделирования
! (в т.ч. банковских очередей!)

Class Account;
Begin
    Real balance;

    Procedure Deposit(amount);
        Real amount;
    Begin
        balance := balance + amount
    End;
End;
```

Smalltalk-80

```
"Всё — объекты, общение —
 сообщениями"

Object subclass: #Account
    instanceVariableNames:
        'balance'
    classVariableNames: ''

"Пополняем счёт — отправляем
 объекту СООБЩЕНИЕ:"
anAccount deposit: 100.

deposit: amount
    balance := balance + amount
```

Уже в 1967 видны класс, поле и метод. В Smalltalk не «вызывают метод», а отправляют объекту сообщение — зародыш полиморфизма.

Чем вдохновлялись авторы: Кристен Нюгор и Оле-Йохан Даль

Зачем на самом деле придумали ООП

- 1962–1967, Норвегия, Норвежский вычислительный центр. Задача — компьютерные СИМУЛЯТОРЫ: порты, производственные линии, очереди
- Вопрос плановиков: «что будет, если у порта 3 крана вместо 2?» — ответ должна дать модель
- Математическая база: метод Монте-Карло и теория массового обслуживания — наука об очередях (как в банке!)
- Проблема: Fortran и Algol описывали ПРОЦЕДУРЫ. Нюгор хотел описывать СУЩНОСТИ: корабли, доки, клиенты в очереди
- **Решение: объект = маленькая самостоятельная «машина» со своими данными и правилами; время идёт шагами, объекты взаимодействуют**
- Побочный эффект: приём оказался универсальным — Симула повлияла на C++, Java, C# и Python

ООП родилось не из «красоты кода», а из желания описывать мир так, как он устроен: из взаимодействующих сущностей.

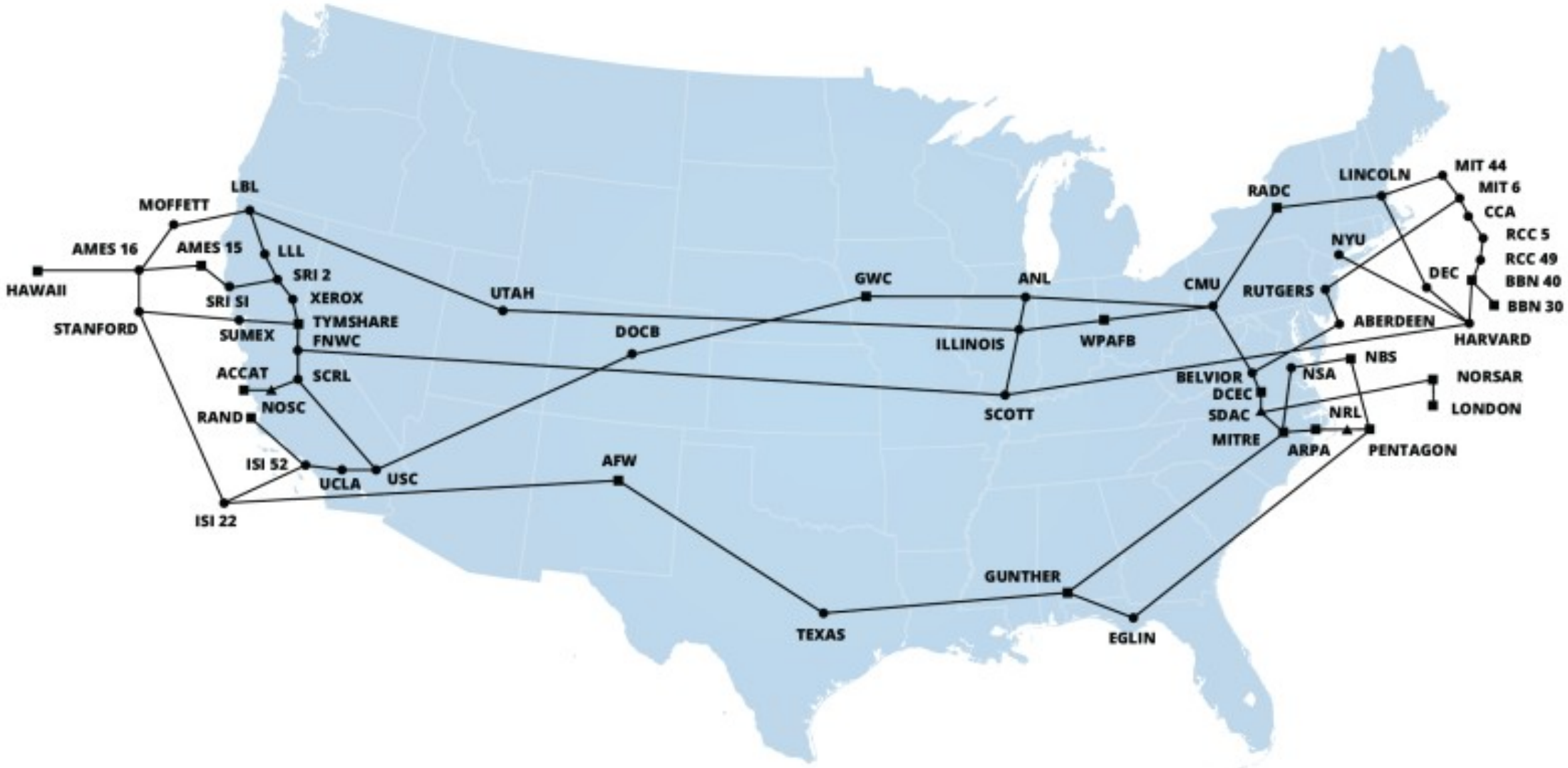
Чем вдохновлялся Алан Кей: биология вместо математики

Smalltalk, 1970-е

- 1970-е, Xerox PARC (США). Алан Кей создаёт Smalltalk. Странная цель: язык, понятный даже ДЕТЯМ
- Главное вдохновение — БИОЛОГИЯ: организм состоит из клеток; клетки общаются сигналами, не залезая друг к другу внутрь
- **Отсюда идея: объекты обмениваются СООБЩЕНИЯМИ, не зная внутреннего устройства друг друга — так рождаются инкапсуляция и полиморфизм**
- Ещё источники: Sketchpad Ивана Сазерленда (1963) — фигуры-объекты, которые копируются и меняются; Lisp; ARPANET — сеть независимых узлов
- Формула Кея: «много маленьких компьютеров, общающихся сообщениями» — интерпретатор Smalltalk был таким «компьютером»
- Урок: принципы ООП — не догмы, а попытка скопировать у живых систем их способ выживать через простоту частей

Вендинговый автомат — прямой наследник клеточной идеи Кея.

1977 ARPANET



Одна и та же идея в разных языках

Класс Account на трёх языках — найдите сходство

C++ (1985)

```
class Account {
public:
    Account(double b)
        : balance(b) {}
    void deposit(double a) {
        balance += a;
    }
private:
    double balance;
};
```

Java (1995)

```
public class Account {
    private double balance;

    public Account(double b) {
        balance = b;
    }
    public void deposit(double a) {
        balance += a;
    }
}
```

Python (1991)

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    def deposit(self, amount):
        self._balance += amount
```

Приватный balance и конструктор — везде. Анатомия одинаковая, Python делает то же самое минимумом кода.

C# (2000, Microsoft): ООП по-взрослому

```
// Классы, интерфейсы, свойства, строгая типизация
public class Account
{
    // свойство: читать можно всем,
    // менять — только внутри класса
    public decimal Balance { get; private set; }

    public Account(decimal balance)
    {
        Balance = balance;
    }

    public bool Deposit(decimal amount)
    {
        if (amount <= 0) return false;
        Balance += amount;
        return true;
    }
}

Account acc = new Account(1000m);
acc.Deposit(500m);
```

Balance { get; private set; } — это ровно наш @property: чтение как атрибут, запись только под контролем класса.

Go (2009, Google): ООП без классов вообще

```
// В Go НЕТ классов, наследования и super().
// Есть структуры данных — и методы к ним.
type Account struct {
    balance float64
}

func (a *Account) Deposit(amount float64) bool {
    if amount <= 0 {
        return false
    }
    a.balance += amount
    return true
}

// Интерфейс — просто набор методов.
// Слово "implements" не нужно: подходит
// любой тип, у которого есть Deposit
type Depositable interface {
    Deposit(float64) bool
}
```

Go — ответ индустрии на боли ООП: композиция вместо наследования..

ООП и банковская сфера: давняя история

- 1960–70-е: банки — первые «большие клиенты» разработчиков
- **Счёт, клиент, транзакция, карта — идеально ложатся в объекты**
- 1980-е: банковские АБС пишут на C++ / Smalltalk
- 1990-е — Java и «банк на дискете»: дебетовые карты, банкоматы — всё на ООП
- Сегодня: Python — в финтехе, антифрод-системах, расчётах рисков

Банковская модель «счёт => операция => проводка» — это классическая объектная модель предметной области.

ЧАСТЬ 1

Классы и объекты

Чертёж и печенье: шаблон против экземпляра



Словарь ООП

- Класс (class) — чертёж, шаблон, рецепт, форма. Ещё не объект, а описание
- Объект / экземпляр (instance) — «готовый кекс»: конкретный экземпляр класса
- Атрибут — данные объекта: имя, возраст, баланс
- Метод — функция внутри класса: действие объекта
- self — ссылка на конкретный объект (притяжательное местоимение первого лица в речи объекта)

int, str, list — это классы. 1, «abc», [1,2,3] — объекты (экземпляры).

Первый класс

```
class Cat:
    """Класс, описывающий кота. Мяу!"""

    def __init__(self, name, age):
        self.name = name      # атрибуты
        self.age = age

    def meow(self):           # метод
        print(f"{self.name} говорит: Мяу!")

barsik = Cat("Барсик", 4)     # создание объекта
barsik.meow()                 # Барсик говорит: Мяу!
print(barsik.name)            # Барсик
barsik.age = 5                # атрибут можно менять
```

__init__ вызывается автоматически при создании объекта. Метод вызывается БЕЗ self — Python подставит объект сам.

Что такое self

- Первый параметр любого метода — всегда self (контекстный объект)
- **Python под капотом превращает barsik.meow() в Cat.meow(barsik)**
- Без self данные уйдут в локальную переменную и исчезнут после метода
- У каждого объекта — свои атрибуты: своя копия name, age и т.д.
- Методы хранятся в классе в одном экземпляре и разделяются всеми объектами

1000 котов = 1000 копий атрибутов, но только 1 копия метода meow() в памяти.

Что такое bound method

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        return f'{self.name} says Meow!'

cat1 = Cat('Tom')
print(cat1.meow())
print(id(cat1))
print(id(cat1.meow.__func__))

cat2 = Cat('Whiskers')
print(cat2.meow())
print(id(cat2))
print(id(cat2.meow.__func__))
```

```
Tom says Meow!
124711866927152
124711866696384

Whiskers says Meow!
124711866572368
124711866696384
```

Когда вы пишете cat1.meow, Python не возвращает вам саму функцию из класса. Он создаёт bound method (связанный метод) — специальный объект-обёртку, который «помнит» и функцию, и объект, к которому она привязана.

Атрибуты можно добавлять «на лету»

```
class Fruit:
    pass

apple = Fruit()
banana = Fruit()

apple.name = "Яблоко"
apple.weight = 150

banana.name = "Банан"
banana.origin = "Эквадор"   # только у банана!

print(apple.origin)
# AttributeError: у apple нет атрибута origin
```



Гибко, но опасно: программа падает, если забыли создать атрибут. Решение — инициализатор __init__.
Объекты в Python хранят свои атрибуты в специальном словаре __dict__, который можно свободно пополнять

ЧАСТЬ 2

Инкапсуляция

Принцип №1 · и первый банковский кейс

Четыре столпа ООП

- **Инкапсуляция — скрыть внутренности, дать наружу безопасный интерфейс**
- Наследование — новый класс на базе старого: переиспользуем и расширяем
- Полиморфизм — один интерфейс, разное поведение у разных объектов
- Абстракция — оставить только существенное, детали скрыть

Разберём каждый на примерах из банковской предметной области.

Инкапсуляция: идея

- Аналогия: кофе-машина. Кнопки «эспрессо»/«капучино» — интерфейс. Нагреватели и моторы — скрыты
- Внутренние данные класса — «приватные», снаружи не трогаем
- Доступ — только через методы класса (геттеры и сеттеры)
- **Методы содержат проверки: нельзя «ввести отрицательную сумму» или «снять больше, чем есть»**



Сокрытие данных + доступ через методы = защита от некорректного использования.

Кейс «Банк»: как НЕ надо

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

account = BankAccount(1000)

# кто угодно может «напечатать» деньги:
account.balance = 999_999_999
print(account.balance)   # 999999999

# или обнулить чужой счёт:
account.balance = 0
```

Публичный атрибут = никаких гарантий. В банковской системе это — критическая уязвимость.

Кейс «Банк»: правильный интерфейс

```
class BankAccount:
    def __init__(self, initial_balance):
        self._balance = initial_balance

    def deposit(self, amount):
        if amount <= 0:
            return False
        self._balance += amount
        return True

    def withdraw(self, amount):
        if 0 <= amount <= self._balance:
            self._balance -= amount
            return True
         return False

    def get_balance(self):
        return self._balance
```

Соглашения о приватности + инф. безопасность

Связь с инфобезопасностью

- _balance — одно подчёркивание: «внутреннее дело, не трогай» (соглашение)
- __balance — два подчёркивания: Python подменяет имя (_BankAccount__balance)
- Настоящей приватности в Python нет — есть дисциплина и соглашения

Принцип наименьших привилегий (least privilege): код получает доступ только к тому, что ему необходимо.

Публичный атрибут баланса — как дверь в хранилище без замка.

Инкапсуляция — та же идея, что firewall и ролевой доступ в банках: **интерфейс узкий, проверки строгие.**

Современный способ: @property

```
class BankAccount:
    def __init__(self, initial_balance):
        self._balance = initial_balance

    @property
    def balance(self):          # читается КАК АТРИБУТ
        return self._balance

    @balance.setter
    def balance(self, value):   # но присваивание проверяется!
        if value < 0:
            raise ValueError("Баланс не может быть отрицательным")
        self._balance = value

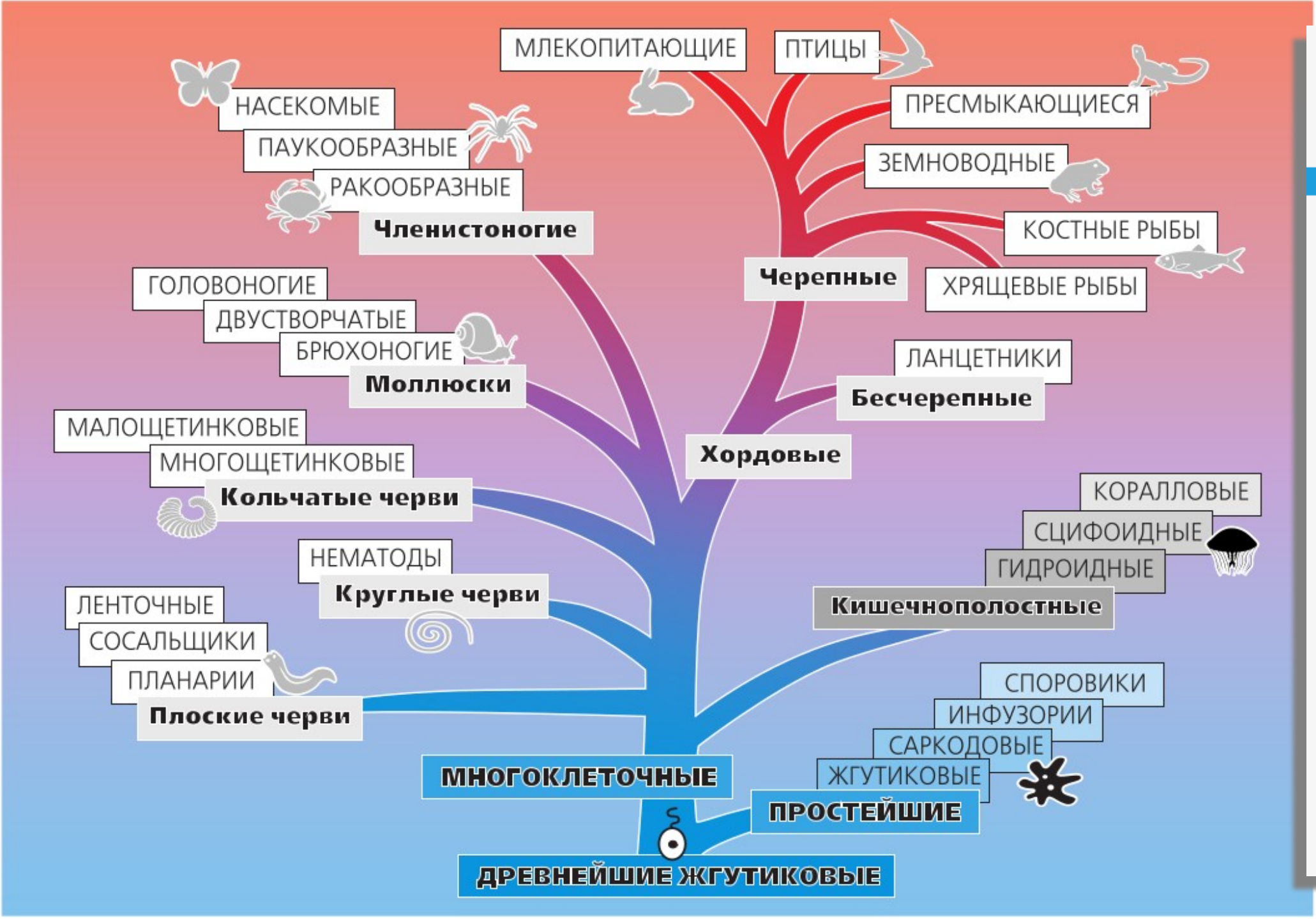
account = BankAccount(1000)
print(account.balance)   # 1000 — без скобок!
account.balance = -500   # ValueError
```

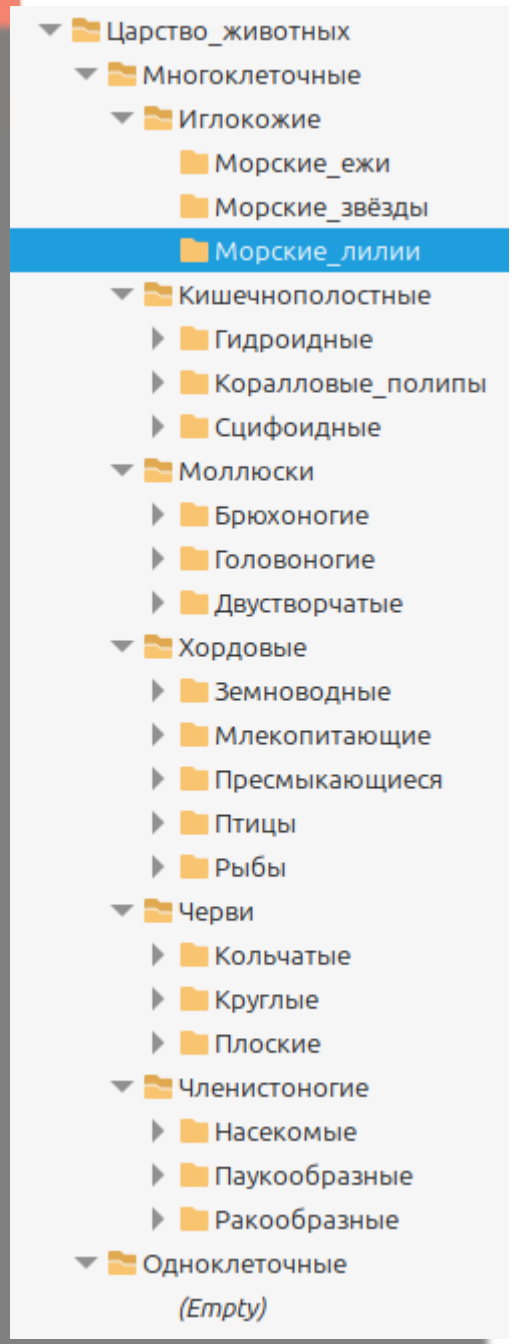
ЧАСТЬ 3

Наследование

Принцип №2 · иерархии счетов и карт




ДРЕВНЕЙШИЕ ЖГУТИКОВЫЕ
МНОГОКЛЕТОЧНЫЕ
ПРОСТЕЙШИЕ
ЖГУТИКОВЫЕ
САРКОДОВЫЕ
ИНФУЗОРИИ
СПОРОВИКИ
Кишечнополостные
ГИДРОИДНЫЕ
СЦИФОИДНЫЕ
КОРАЛЛОВЫЕ
Плоские черви
ПЛАНАРИИ
СОСАЛЬЩИКИ
ЛЕНТОЧНЫЕ
Круглые черви
НЕМАТОДЫ
Кольчатые черви
МНОГОЩЕТИНКОВЫЕ
МАЛОЩЕТИНКОВЫЕ
Моллюски
БРЮХОНОГИЕ
ДВУСТВОРЧАТЫЕ
ГОЛОВОНОГИЕ
Членистоногие
РАКООБРАЗНЫЕ
ПАУКООБРАЗНЫЕ
НАСЕКОМЫЕ
Бесчерепные
ЛАНЦЕТНИКИ
Черепные
ХРЯЩЕВЫЕ РЫБЫ
КОСТНЫЕ РЫБЫ
ЗЕМНОВОДНЫЕ
ПРЕСМЫКАЮЩИЕСЯ
ПТИЦЫ
МЛЕКОПИТАЮЩИЕ
Хордовые


Царство_животных
Многоклеточные
Иглокожие
Морские_ежи
Морские_звёзды
Морские_лилии
Кишечнополостные
Гидроидные
Коралловые_полипы
Сцифоидные
Моллюски
Брюхоногие
Головоногие
Двустворчатые
Хордовые
Земноводные
Млекопитающие
Пресмыкающиеся
Птицы
Рыбы
Черви
Кольчатые
Круглые
Плоские
Членистоногие
Насекомые
Паукообразные
Ракообразные
Одноклеточные
(Empty)

Наследование: базовый и дочерний классы

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def speak(self):
        return "*неопределённый звук*"


class Dog(Animal):              # Dog наследует Animal
    def speak(self):            # переопределили
        return f"{self.name}: Гав-гав!"

class Cat(Animal):
    def speak(self):            # и снова переопределили
        return f"{self.name}: Мяу!"
```

Собака и кот получают name и age «бесплатно», а speak() — со своим поведением.

super() — связь с родителем

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def speak(self):
        return "*неопределённый звук*"

class Dog(Animal):
    def __init__(self, name, age, breed):
        super().__init__(name, age)   # вызов __init__ родителя
        self.breed = breed            # + своё расширение

class Square(Rectangle):
    def __init__(self, size):
        super().__init__(size, size)  # расширение родительского метода
```

Расширение — вызываем родителя и добавляем своё. Переопределение — пишем метод заново.

Кейс «Банк»: иерархия счетов

```
class Account:
    def __init__(self, number, owner, balance=0):
        self.number = number
        self.owner = owner
        self.balance = balance

    def withdraw(self, amount): ...   # общая логика

class CreditAccount(Account):          # кредитный счёт
    def __init__(self, number, owner, limit):
        super().__init__(number, owner)
        self.limit = limit             # + кредитный лимит

class DepositAccount(Account):         # вклад
    def __init__(self, number, owner, rate):
        super().__init__(number, owner)
        self.rate = rate               # + процентная ставка
```

Общее — в базовом классе. Различия — в наследниках. Новый продукт банка = новый подкласс.

Множественное наследование

```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def speak(self):
        return "*неопределённый звук*"

class Flyable:
    def fly(self):
        return f"{self.name} летит!"

class Swimmable:
    def swim(self):
        return f"{self.name} плывёт!"

class Duck(Animal, Flyable, Swimmable):
    def speak(self):
        return f"{self.name}: Кря-кря!"

donald = Duck("Дональд", 5)
donald.fly()    # метод от Flyable
donald.swim()   # метод от Swimmable
```



Осторожно: при совпадении имён методов порядок разрешения — слева направо (MRO).

Коллизии: чем ООП может навредить

Честный разговор о цене наследования

- Ромбовидное наследование (diamond problem): D наследует B и C, оба — от A. Чей метод A вызвать? Python решает через MRO, но в C++ это была реальная боль
- **Хрупкий базовый класс (fragile base class): поменял «внутренности» родителя — незаметно сломал всех наследников**
- Конфликт имён: два базовых класса с методом run() — какой достанется наследнику, зависит от порядка в скобках
- Наследование пробивает инкапсуляцию: наследник знает внутренности родителя и зависит от них
- God Object: класс на 5000 строк, который «всё знает и всё умеет» — типичный итог неограниченного роста иерархии
- **Лекарство: композиция вместо наследования, плоские иерархии, абстрактные классы-контракты**

«Наследование — самая сильная связь в ООП, а значит — самая опасная» (Бертран Мейер, автор Eiffel).

The Diamond Problem

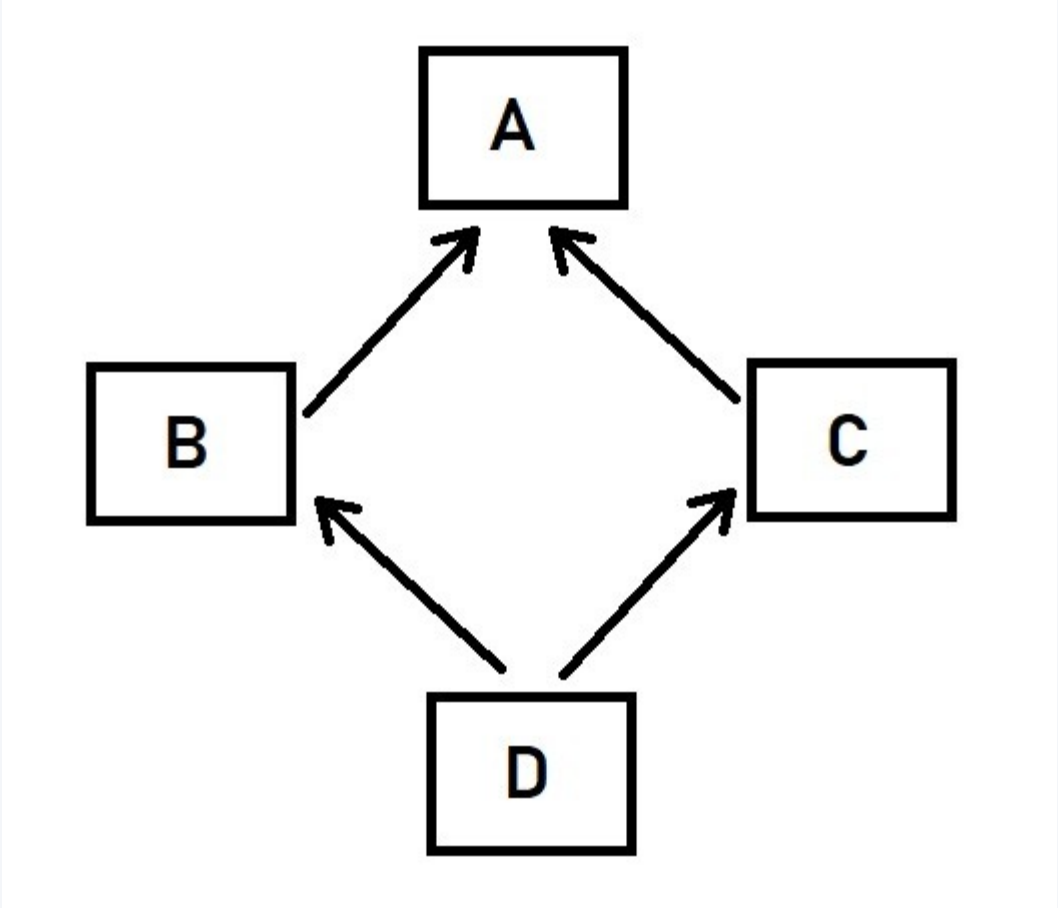
```
class A:
    def who_am_i(self):
        print("Class A")

class B(A):
    def who_am_i(self):
        print("Class B")

class C(A):
    def who_am_i(self):
        print("Class C")

class D(B, C):
    pass

obj = D()
obj.who_am_i()
```




```
print(D.__mro__)
# (<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>)
```

ЧАСТЬ 4

Полиморфизм

Принцип №3 · один интерфейс — много поведений

Полиморфизм: одна команда — разные действия

```
class Dog:
    def speak(self): return "Гав-гав!"

class Cat:
    def speak(self): return "Мяу!"

class Parrot:
    def speak(self): return "Кеша хороший!"

zoo = [Dog(), Cat(), Parrot()]
for animal in zoo:
    print(animal.speak())
```

Учитель говорит «запишите тему урока»: кто-то пишет ручкой, кто-то — стилусом. Команда одна, действия разные.

Утиная типизация (duck typing)

- **«Если что-то ходит как утка и крякает как утка — это утка»**
- Python не проверяет класс объекта — только наличие нужных методов
- Функция print_area() работает с ЛЮБОЙ фигурой, у которой есть метод area()
- Проверки isinstance() нужны редко — только когда поведение правда зависит от типа

Антипаттерн: цепочка if isinstance(shape, Circle) ... elif isinstance(shape, Square) ... — это ломает полиморфизм.

Строковое представление: __str__ и __repr__

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def __str__(self):              # для людей
        return f"Круг радиуса {self.radius}"

    def __repr__(self):             # для программистов
        return f"Circle(radius={self.radius})"

c = Circle(10)
print(c)        # Круг радиуса 10
print([c])      # [Circle(radius=10)]
print(str(c))   # Круг радиуса 10
print(repr(c))  # Circle(radius=10)
```

__str__ — красиво, __repr__ — информативно. Идеал: repr() можно скопировать и выполнить как код.

ЧАСТЬ 5

Магические методы

Перегрузка операторов · договор с интерпретатором

Перегрузка операторов: суть

- **Операторы — не «встроенные штуки», а вызовы спецметодов**
- x + y → x.__add__(y)
- len(obj) → obj.__len__()
- obj[i] → obj.__getitem__(i)
- Методы с двойным подчёркиванием (__add__, __eq__, __lt__…) — dunder-методы
- **Определил метод — соответствующий синтаксис заработал с твоими объектами**

Шпаргалка dunder-методов (1/2): арифметика и сравнение

Для «укороченных» операций +=, -= есть «i-версии»: __iadd__, __isub__ и т.д.

Операция	Метод	Что происходит
x + y	`__add__`	сложение (правый вариант — __radd__)
x - y	`__sub__`	вычитание
x * y	`__mul__`	умножение
x / y	`__truediv__`	деление
x // y	`__floordiv__`	целочисленное деление
x % y	`__mod__`	остаток от деления
x ** y	`__pow__`	возведение в степень
divmod(x, y)	`__divmod__`	частное и остаток
-x	`__neg__`	унарный минус
abs(x)	`__abs__`	модуль числа
x == y	`__eq__`	равенство
x != y	`__ne__`	неравенство
x < y	`__lt__`	меньше
x <= y	`__le__`	меньше или равно
x > y	`__gt__`	больше
x >= y	`__ge__`	больше или равно
hash(x)	`__hash__`	хэш для set и dict

Шпаргалка dunder-методов (2/2): коллекции и преобразования

Полный список: docs.python.org/3/reference/datamodel.html#special-method-names

Операция	Метод	Что происходит
len(obj)	`__len__`	длина коллекции
obj[i]	`__getitem__`	чтение по индексу/ключу (и срезы)
obj[i] = v	`__setitem__`	запись по индексу/ключу
del obj[i]	`__delitem__`	удаление элемента
x in obj	`__contains__`	проверка вхождения
for x in obj	`__iter__`	итерация (план Б — __getitem__)
next(it)	`__next__`	следующий элемент итератора
str(obj)	`__str__`	строка для людей (print, f-строки)
repr(obj)	`__repr__`	строка для программистов (списки, отладка)
int(obj)	`__int__`	преобразование в int
float(obj)	`__float__`	преобразование в float
bool(obj)	`__bool__`	истинность (план Б — __len__)
obj()	`__call__`	вызов объекта как функции
with obj:	`__enter__ / __exit__`	контекстный менеджер (файлы, БД)
obj.attr	`__getattr__`	доступ к несуществующему атрибуту

__add__: складываем свои объекты

```
class Money:
    def __init__(self, amount, currency="RUB"):
        self.amount = amount
        self.currency = currency

    def __add__(self, other):
        if self.currency != other.currency:
            raise ValueError("Нельзя сложить разные валюты!")
        return Money(self.amount + other.amount, self.currency)

    def __str__(self):
        return f"{self.amount:.2f} {self.currency}"

a = Money(100, "RUB")
b = Money(50, "RUB")
print(a + b)     # 150.00 RUB
```

Возвращай НОВЫЙ объект, не меняя исходные (a + b не должно «портить» a).

NotImplemented и «правые» методы

```
class Money:
    ...
    def __add__(self, other):
        if not isinstance(other, Money):
            return NotImplemented    # «попробуй у другого объекта»
        ...

    def __radd__(self, other):       # для записи: 100 + money
        return self.__add__(other)
```

Если money + 100 не сработал, Python пробует 100.__radd__(money) наоборот.

Операторы сравнения

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def __eq__(self, other):     # ==
        return (self.grade, self.name) == (other.grade, other.name)

    def __lt__(self, other):     # <
        return (self.grade, self.name) < (other.grade, other.name)

students.sort()                  # использует __lt__ под капотом
print(max(students))             # тоже
```

Много критериев сравнения? Сравнивай КОРТЕЖИ — сначала важный критерий, потом следующий.

Лень — двигатель прогресса: @total_ordering

```
from functools import total_ordering

@total_ordering
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def __eq__(self, other):
        return self.grade == other.grade

    def __lt__(self, other):
        return self.grade < other.grade
    # __le__, __gt__, __ge__ создадутся САМИ
```

Достаточно __eq__ + один метод сравнения. Остальные декоратор выведет автоматически.

Свой класс — как коллекция

```
class Playlist:
    def __init__(self, tracks):
        self._tracks = list(tracks)

    def __len__(self):          return len(self._tracks)
    def __getitem__(self, i):   return self._tracks[i]   # и срезы!
    def __contains__(self, t):  return t in self._tracks

p = Playlist(["Muse", "Green Day"])
print(len(p))          # 2
print(p[0])            # "Muse"
print(p[1:])           # срез работает
print("Muse" in p)     # True
for track in p: ...    # перебор тоже
```

Реализуй дандер-методы — и твой объект ведёт себя как список.

Перегрузка операторов + инф. безопасность

Связь с инфобезопасностью

- __eq__ и __hash__ определяют поведение в set и dict — ошибки здесь = уязвимости в логике
- **Правило: если a == b, то hash(a) == hash(b). Иначе — коллизии в хэш-таблицах**
- Не делай операторы неинтуитивными: a + b должно вести себя ожидаемо
- Всегда проверяй типы и возвращай NotImplemented

Банковский пример: если Transaction.__eq__ не учитывает валюту, платёжная система может «не заметить»

дубль транзакции. А если __hash__ «плавает» — словари кэша разваливаются под нагрузкой.

Корректность dunder-методов — это часть надёжности системы.

ЧАСТЬ 6

Исключения

Ошибки под контролем + инф. безопасность

Шаблон try-except-finally

```
try:
    x = int(input("Введи число: "))
    y = 10 / x
except ValueError:
    print("Это не число.")
except ZeroDivisionError:
    print("На ноль делить нельзя.")
else:
    print(f"Результат: {y}")
finally:
    print("Выполнится всегда.")
```

else — если ошибок не было. finally — всегда (закрыть файл, освободить ресурс).

Свои исключения

```
class GameError(Exception):
    pass

class NotEnoughHealthError(GameError):
    pass

def attack(hp, cost):
    if hp < cost:
        raise NotEnoughHealthError(f"Нужно {cost} HP, а у тебя {hp}")

try:
    attack(5, 10)
except NotEnoughHealthError as e:
    print("Ошибка игры:", e)
```

Иерархия исключений = иерархия классов. Ловим конкретное, а не всё подряд.

Исключения + инф. безопасность

Связь с инфобезопасностью

- **Никогда не делай except: pass — глотание ошибок маскирует атаки**
- Не выводи пользователю внутренности: stack trace показывает структуру системы
- **Логируй детали — показывай пользователю общее сообщение**
- Валидация входных данных — первая линия защиты от инъекций и подделок

Плохо: except Exception: pass → атака прошла незамеченной

Плохо: print(traceback) пользователю → раскрытие внутренней структуры

Хорошо: лог полный — ответ пользователю нейтральный: «Операция отклонена»

Хорошо: проверка суммы, валюты, лимитов — до выполнения операции

@classmethod и @staticmethod

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    @classmethod
    def from_birth_year(cls, name, year):   # работает с классом
        return cls(name, 2026 - year)

    @staticmethod
    def is_adult(age):                       # не трогает ни класс, ни объект
        return age >= 18

p = Person.from_birth_year("Анна", 2000)
print(Person.is_adult(p.age))   # True
```

@classmethod — альтернативный конструктор. @staticmethod — служебная функция рядом с классом.

Абстракция: абстрактные классы

```
from abc import ABC, abstractmethod

class Shape(ABC):                    # «чертёж для чертежей»
    @abstractmethod
    def area(self):
        pass                          # реализации НЕТ

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):                   # обязаны реализовать!
        return 3.14 * self.r ** 2

# s = Shape()        # Ошибка! Shape нельзя создать
c = Circle(5)        # а Circle — можно
```

Абстрактный класс задаёт контракт: наследники ОБЯЗАНЫ реализовать методы. Иначе — ошибка сразу.

Антипаттерны: чего делать НЕ стоит

- isinstance-цепочки там, где достаточно общего интерфейса
- Изменять объекты внутри __add__ вместо возврата нового
- Класть неинтуитивный смысл в операторы (+, который удаляет?)
- except: pass — молчаливая обработка ошибок
- Игнорировать __repr__ — потом часами ищешь объект в отладчике
- Забывать про self и NotImplemented

Золотое правило: код читают чаще, чем пишут. Пиши для читателя.

SOLID (1/2): S, O, L — три принципа, которые спасят ваш код

Эвристики хорошего дизайна, а не законы

- S — Single Responsibility («одна ответственность»): класс решает одну задачу. Плохо: Report, который считает итоги, печатает PDF и шлёт email. Хорошо: Report + PdfPrinter + EmailSender — каждый меняется независимо
- O — Open/Closed («открыт для расширения, закрыт для изменения»): новый тип платежа = новый класс, а не новая ветка if в старом коде. Мы уже знаем как: полиморфизм + абстрактные классы
- **L — Liskov Substitution («подстановка Лисков»): наследник должен заменять родителя БЕЗ сюрпризов. Код, работающий с Rectangle, не должен ломаться, если получит Square**

Автор аббревиатуры — Роберт Мартин («Дядя Боб»), 2000-е. Идеи старше: принципы собирались годами из опыта индустрии.

SOLID (2/2): L, I, D — на примерах кода

```
# L — подстановка Лисков: наследник НЕ ломает ожидания родителя
class Rectangle:
    def set_width(self, w):  self.width = w
    def set_height(self, h): self.height = h

class Square(Rectangle):     # ❌ set_width сломает и "высоту"
    def set_width(self, w):
        self.width = self.height = w

# I — разделение интерфейсов: узкие роли вместо "всеядного" класса
class Coder:     def write_code(self): ...
class Designer:  def draw_ui(self): ...

# D — инверсия зависимостей: зависим от абстракции, не от конкретики
class Report:
    def __init__(self, db):  # любая БД: SQLite, PostgreSQL, мок в тестах
        self.db = db
```

На первых порах хватит S и O. SOLID — компас, а не догма: не превращайте простой код в храм абстракций.

Когда ООП избыточно: честность важнее пиетета

Не каждая задача нуждается в классах

- Скрипт на 100–200 строк, ETL-конвейер, разовый парсер — функций и dict более чем достаточно
- **Класс без состояния и наследников — это функция в костюме: пишите функцию**
- «Пирамида абстракций»: AbstractFactoryManagerFactory — усложнение ради усложнения (YAGNI: «вам это не понадобится»)
- Data Science: pandas DataFrame и numpy — данные «плоско», быстрее и проще, чем 50 классов-обёрток
- Тренд индустрии: Go и Rust — композиция и данные вместо глубоких иерархий (см. слайд про Go)
- **Правило большого пальца: начинай с функций; класс появляется, когда заводится СОСТОЯНИЕ и повторное использование**

Плохой ООП-код хуже хорошего процедурного. Инструмент под задачу, а не под религию.

Итоги занятия

- Класс — чертёж, объект — экземпляр. __init__ заполняет, self указывает
- Инкапсуляция — доступ через безопасный интерфейс (банковский кейс)
- Наследование + super() — общее вверх, различия вниз. Но помните о коллизиях
- Полиморфизм — один интерфейс, разное поведение. Утиная типизация
- **Dunder-методы — договор с интерпретатором: +, ==, len(), [] работают с твоими классами**
- Исключения — контролируемые ошибки. Никогда except: pass
- SOLID — компас дизайна. ООП — инструмент, а не религия

Инфобезопасность: принцип наименьших привилегий, валидация ввода, нейтральные сообщения об ошибках — встроены в хороший ООП-дизайн.

Мини-практика (дома или в конце занятия)

5 задач · 20–30 минут

- Класс BankAccount: инкапсуляция, deposit/withdraw с проверками, @property
- Иерархия: Account → CreditAccount, DepositAccount. super() обязателен
- Полиморфизм: список фигур → общий цикл с area() и perimeter()
- Класс Money с __add__, __eq__, __lt__ и защитой от разных валют
- Свои исключения: InsufficientFundsError, InvalidAmountError

Куда дальше

- Официальная документация Python — раздел Data Model (все dunder-методы)
- docs.python.org/3/tutorial/classes.html — классы и множественное наследование
- Модули functools.total_ordering, abc — стандартная библиотека
- Что ещё: паттерны проектирования, миксины, протоколы

Всё, что сегодня разобрали — фундамент. Паттерны проектирования — следующий уровень.